

LESSONS — written once, not re-typed every handoff

These do not change with the corpus, so they do not belong in a status print and they do not belong in a dated handoff. Each one cost hours. Referenced by name from the division ledgers.

1. A silent default looks exactly like a working feature

The lead search never ran in a real game across **four** layers of bug. The post-KO search logged every turn and never once decided. Both reported success the entire time.

This is the failure mode this project actually has, and it is invisible to every automated check: a head-to-head, an exploitability search and a prediction score all compare two bots that **share** the blind spot, so a missing capability cancels out exactly.

Make every fallback loud. A default that fires silently is indistinguishable from a feature that works.

2. Search amplifies model error

A maximiser seeks out the lines its model is most optimistic about — which are precisely the lines it is most wrong about. This is why R4 came back negative on the broken engine and positive on the fixed one with the same search and the same flags.

A search is worth exactly what its model is worth. It follows that improving the leaf beats improving the depth, every time, until the leaf is calibrated.

3. Usage counts are sheet counts

Blaze reads 4,585 uses and is worthless: 30 of 54 entries are a Charizard that megas into Drought on turn one, so the ability never fires. Ice Scales, Filter, Aerilate, Prism Armor, Punk Rock and Ripen read zero.

A count of how often something appears on a sheet is not a count of how often it matters.

4. Every derivation over-matches on the first try

- `refusesStatusMoves` caught Telepathy and Wonder Guard
- `speedOnItemLoss` caught Sticky Hold
- `failsIfTargetNotAttacking` caught Quick Guard, Wide Guard and Round

Print what a new tag matched before wiring it. Every time. The list is always longer than expected and the extras are always plausible.

5. My own probes were wrong about fifteen times, always toward a comfortable answer

Three rules that came out of it:

- **Clear the control explicitly.** The first Choice Scarf probe compared a Scarf against a Basculegion that `buildMon` had already given a Scarf, and reported the engine broken.
- **Test the outcome, not the classification.** Whether the code labels something correctly is not whether the thing happens.

- **Identical results across a varied knob mean the knob is unwired.** Not that the knob does not matter.

Two more, added 2026-08-04 after five further cases in one session. The count is now roughly twenty and it is no longer bad luck; it is a property of how probes get written.

- **STAGE IT AGAINST A BODY THAT CAN SHOW THE EFFECT.** Three of the five failed on this alone. The redirection "bug" that led this session's headline for six hours was a probe firing **Dragon** Claw at a **Fairy** type: the redirect worked perfectly, moved the attack onto something immune, and both arms read 0. The `typeImmunity` wire that sat red for two days staged Volt Absorb on a **Garchomp** — Dragon/Ground, already immune to Electric with no ability at all — and *also* left the absorber holding Protect, so it blocked the hit. Two independent reasons that arm could never pass whatever the engine did. And a heal probe on a full-HP body reads `0 → 0` forever. **Ask what the target would do if the mechanic did not exist. If the answer is "the same thing", the probe is worthless.**
- **A FAKE INPUT MUST CARRY THE FIELD THE ENGINE ACTUALLY READS.** A hand-built move object with `cat: 'Physical'` is not a physical move: `medicham2-browser.js:511 is const phys = mv.c === 'P'`. With `mv.c` absent, `phys` is false for *everything* — which made the sand boost fire where it must not and stopped the snow boost firing where it must, and produced a confident report that **both were broken in opposite directions**. Both were correct. A `mon.moves` entry is a **string**, not an object, so the lookup that was supposed to fetch a real move silently returned nothing and fell through to the fake one. **Prefer a real object from the real path. If you must synthesise one, read the consumer first and match the field names it tests, not the ones that read well.**

6. Will's domain knowledge beat the data repeatedly

Blaze/mega. Gardevoir-is-Trace. Contrary-Staraptor. "Meganium needs the mega." "Reality is not a 1/4 split."

When he says a number looks wrong, check it before defending it.

7. Never read an interim SPRT

66.7% became 44%. 57.7% became 50%. The bound exists exactly so that you do not have to look, and SPRT is only valid under continuous monitoring because its boundaries were derived for it. Reading a Wilson interval repeatedly and stopping when it clears zero does not inherit that property — it badly inflates the false-positive rate.

If you stop a run at a bound, report the SPRT verdict. Not a p-value computed as though n had been fixed in advance.

8. Reuse the canonical path

Hand-rolling a second body builder produced `buildMon("Scizor") -> null`. `dmgMon` already existed and does it right.

The same instinct is why `provenance.js` derives the artifact graph from source instead of carrying a list, and why `status.js` shells out to `provenance.js` rather than reimplementing its staleness rules.

9. Measure the noise floor before believing an effect

Split one arm in half and measure the spread between the halves. An effect smaller than that spread is not an effect, however clean the number looks.

10. Check the corpus stamps before attributing an effect to a lever

SLOWKING's equilibrium and its named rock-paper-scissors cycle justified an entire architecture. The file was computed on the unfiltered store — 87% bots, forfeits and stubs — one day before the quality filter landed. Nothing on the file said so. Re-run on clean data the cycle disappears entirely.

A JSON file on disk looks equally authoritative whether it was generated this morning or before the filter that made it wrong.